

Data Structures and Algorithms

CS 361

Sorting and Hashing

Ricardo Garcia
`rickwgarcia@unm.edu`

Shrey Poshiya
`sposhiya@unm.edu`

Ethan Hoover
`ehoover@unm.edu`

University of New Mexico

Spring 2026
Due: April 5, 2026

Contents

1	Introduction	2
2	Preliminaries and Methods	3
2.1	Three-Way Merge Sort	3
2.2	Bloom Filter / Password Breach Checker	6
3	Conclusion	10
3.1	Three-Way Merge Sort	10
3.2	Bloom Filter / Password Breach Checker	10
4	Contributions	11
	Bibliography	12

1 Introduction

This paper goes into detail about our implementation of a three-way merge sort and custom FNV-inspired hashing algorithm based bloom filter. All our programs are written in Python. The bloom filter was tested using the first million hashes of the billions available on HaveIBeenPwned.com [2].

Part 1: Three-Way Merge Sort

Merge sort is a classical comparison-based sorting algorithm with an asymptotic running time of $O(n \log n)$. The traditional 2-way merge sort recursively divides an array into two roughly equal halves until subarrays of size one are reached, then merges them back in sorted order. The 3-way merge sort follows the same principle, but divides each subarray into three parts rather than two. This reduces the depth of the recursion tree from $\log_2(n)$ to $\log_3(n)$, though it requires a more complex merge step that simultaneously coordinates three sorted subarrays.

Part 2: Password Breach Checker via Bloom Filter

Bloom filters provide a time-efficient and space-efficient method to determine when an element is *NOT* contained within a set. While it is not possible to deterministically ensure an elements' membership in a set, many problems can benefit from the unique properties of bloom filters (e.g. "username not taken" check). Rather than storing all elements in an array-like data structure and using traditional search algorithms to determine membership, a bloom filter constructs a bit array (of size m) and k -independent hash functions determine the bit-array representation of a specific elements. All bit representations are super-posed to construct a complete binary array representation of the entire set (all collisions during the superposing process are ignored). An inherent drawback of bloom filters are the fact that there is ever-present risk of false positives; this false positive rate can be controlled by tuning the bit array size, m , and the number of independent hash function, k , relative to the original set size.

In this project, we re-create IHaveBeenPwned.com's bloom-filter based search to determine a password's membership in a database of leaked passwords. We re-create this problem on the set of the first 1 million passwords from IHaveBeenPwned.com's leaked password database.

2 Preliminaries and Methods

2.1 Three-Way Merge Sort

Implementation

Traditional merge sort, as seen in Fig. 1, recursively divides an array into two halves until each subarray contains a single element, then merges pairs of subarrays in sorted order, working back up until the entire array is sorted.

The three-way merge sort, shown in Fig. 2, follows the same recursive structure but divides the array into three roughly equal partitions at each level rather than two. The `merge_sort` function computes two split points:

1. $\text{low_mid} = \text{low} + (\text{high} - \text{low}) / 3$
2. $\text{high_mid} = \text{low} + 2 * (\text{high} - \text{low}) / 3$

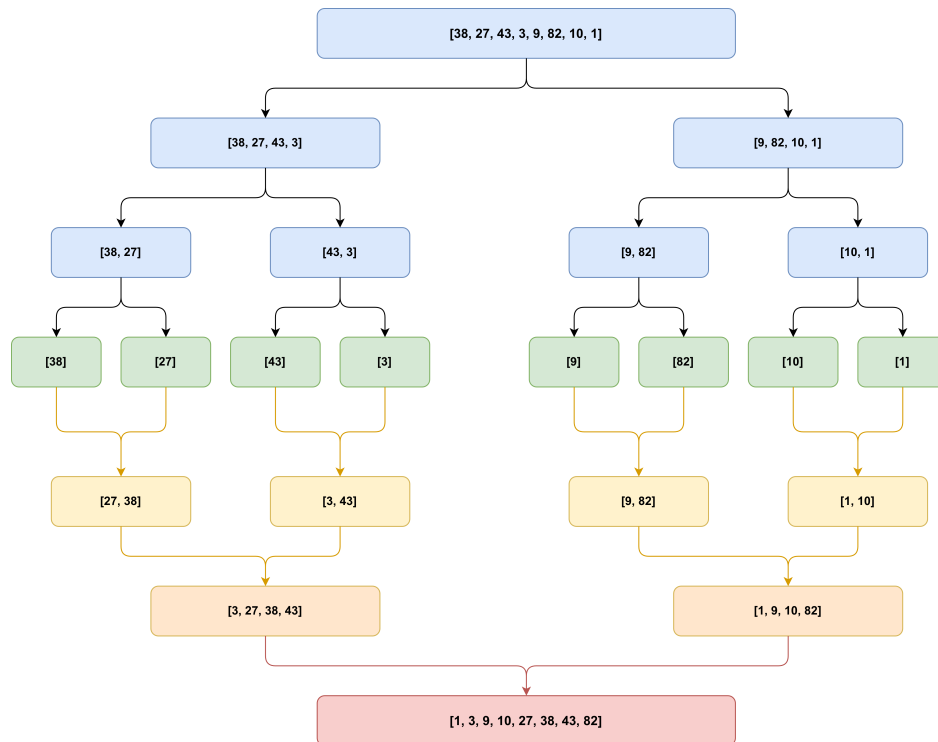


Figure 1: Two-way merge sort divides the input into two halves at each level.

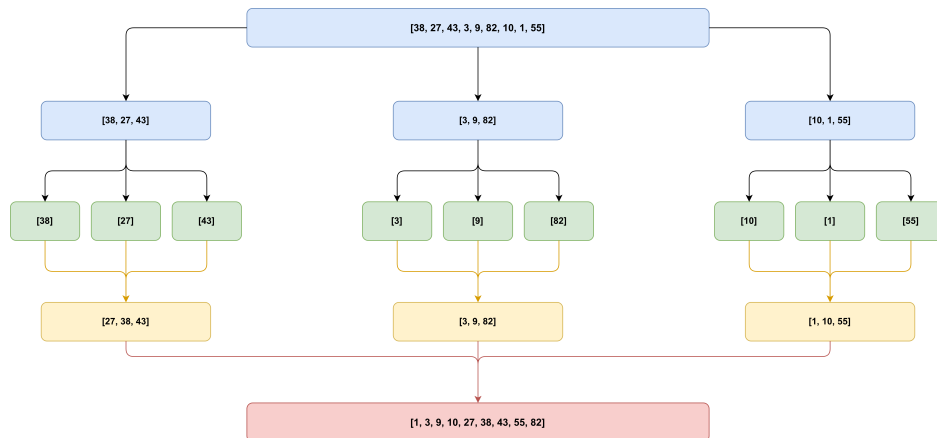


Figure 2: Three-way merge sort divides the input into three roughly equal partitions

The **merge** function carries the added complexity. Where a standard two-way merge maintains two pointers and one comparison per step, the three-way merge maintains three pointers i , j , and p into subarrays A , B , and C respectively. Once any one subarray is exhausted, the remaining two are merged, and any single remaining subarray is copied directly into the output buffer. The final result is copied back into the original array in-place.

Asymptotic Running Time vs Actual Performance

Two way merge sort's recurrence relation can be shown by $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$ which solves to $O(n \log n)$ by the Master Theorem. For the three way merge sort, the recursive calls becomes three, the size of the subproblem becomes $\frac{n}{3}$ and steps changes, which means we now have the recurrence relation for three way merge sort as

$$T(n) = 3T\left(\frac{n}{3}\right) + O(n)$$

Solving by Master Theorem, $a = 3$, $b = 3$, $f(n) = O(n)$, $k = 1$ and $p = 0$. This means that $\log_b a = k$, which results in **Case II**. Since $p > -1$

$$\Theta(n^k \log^{p+1} n) \implies \Theta(n^1 \log^{0+1} n) \implies \Theta(n \log n)$$

To assess whether the actual performance matches the theoretical, we examine how runtime scales as input size doubles. For a $\Theta(n \log n)$ algorithm, doubling n should increase runtime by a factor of

$$\frac{T(2n)}{T(n)} \approx \frac{2n \log(2n)}{n \log n} = 2 \cdot \frac{\log(2n)}{\log n},$$

which is slightly greater than 2 and grows slowly toward 2 as $n \rightarrow \infty$.

Table 1 reports the measured runtimes and the doubling ratio $r_k = T(2^k)/T(2^{k-1})$ for both integer and floating-point inputs for the requested sizes of 2^{20} through 2^{27} . The benchmark

excludes I/O time as input data were generated in memory immediately before each timed sort.

Listing 1: Three-way merge sort benchmark output.

1	-----
2	Size Int (s) Float (s)
3	-----
4	1,048,576 2.6031 2.2090
5	2,097,152 5.2052 5.0486
6	4,194,304 11.3270 11.2970
7	8,388,608 26.7701 23.4131
8	16,777,216 50.1727 49.6396
9	33,554,432 109.3212 110.0315
10	67,108,864 242.4511 234.3279
11	134,217,728 495.4435 493.1873
12	Killed

Table 1: Doubling ratios for three-way merge sort

Input size n	Int (s)	Int ratio	Float (s)	Float ratio
2^{20}	2.6031	—	2.2090	—
2^{21}	5.2052	2.00	5.0486	2.28
2^{22}	11.3270	2.18	11.2970	2.24
2^{23}	26.7701	2.36	23.4131	2.07
2^{24}	50.1727	1.87	49.6396	2.12
2^{25}	109.3212	2.18	110.0315	2.22
2^{26}	242.4511	2.22	234.3279	2.13
2^{27}	495.4435	2.04	493.1873	2.10
2^{28} terminated by the OS due to insufficient memory.				

If we compute the average doubling ratio over the seven measured steps for each data type

$$r_{\text{int}} = \frac{2.00 + 2.18 + 2.36 + 1.87 + 2.18 + 2.22 + 2.04}{7} = \frac{14.85}{7} \approx 2.12,$$

$$r_{\text{float}} = \frac{2.28 + 2.24 + 2.07 + 2.12 + 2.22 + 2.13 + 2.10}{7} = \frac{15.16}{7} \approx 2.17.$$

Table 2: Average doubling ratio comparison

Data type	Sum of ratios	Average ratio
Integer	14.85	2.12
Float	15.16	2.17
Theoretical: ≈ 2.0 – 2.1 for $n \geq 2^{20}$.		

Both averages fall close to the asymptotic prediction of slightly above 2. The slightly higher ratio for floating-point inputs (2.17 vs. 2.12) is consistent with the greater cost of floating points comparisons relative to integer comparisons at large scale.

Memory Issues

On the test machine the highest input size handled was 2^{27} , before the process was killed due to insufficient memory. This is due to merge sort's $O(n)$ space complexity. Unlike in-place sorting algorithms such as quick sort, merge sort allocates a temporary buffer of size n at each merge step to hold the merged result before copying it back into the original array.

Since an integer in python each integer occupies approximately 28 bytes [1]. At $n = 2^{28} \approx 268$ million elements, the input array alone requires roughly

$$2^{28} \times 28 \text{ bytes} \approx 7.5 \text{ GB},$$

before accounting for the auxiliary buffer and the call stack. Combined, these demands exceeded the available RAM on the test machine, causing the process to be terminated.

The recursion depth is not itself the problem. Three-way merge sort recurses to a depth of $\lfloor \log_3 n \rfloor$, which at $n = 2^{28}$ evaluates to

$$\lfloor \log_3 (2^{28}) \rfloor = \left\lfloor 28 \cdot \frac{\log 2}{\log 3} \right\rfloor = \lfloor 28 \times 0.631 \rfloor = 17 \text{ levels}$$

2.2 Bloom Filter / Password Breach Checker

Implementation

Our Bloom Filter is based on the 32-bit FNV hashing function so it relies on a known prime and offset to distribute hashes evenly. The pseudocode for this implementation can be seen in Listing 2. The prime and offset are precisely chosen mathematically to distribute while hashing. The actual values are 16777619 for the prime, and 2166136261 for the offset basis. They are intrinsically related and give well distributed hashes while remaining at 32 bits.

Listing 2: FNV-1a Pseudo Code

```
1 hash = offset_basis
2 for each octet_of_data to be hashed
3     hash = hash xor octet_of_data
4     hash = hash * FNV_Prime
5 return hash
```

The xor is the most fundamental step of a non-cryptographic hashing algorithm. It is a non-destructive operation; no information is lost during the operation of the hash. This guarantees that every single bit of input data dramatically alters the final hash. The seeding problem discussed later is a key example of this, as our fix caused an avalanche effect on the resulting hash.

Our implementation automatically calculates the optimal bit-array size, m , and number of independent hash functions, k using the following equations, relative to our set size, n and the desired false positive rate, p :

$$m = \left\lceil \frac{-n \ln p}{(\ln 2)^2} \right\rceil \quad (1)$$

$$k = \text{round} \left(\frac{m}{n} \cdot \ln 2 \right) \quad (2)$$

In our implementation, we choose $p = 0.01$ (1%).

False Positive Rate

In order to measure the empirical false positive rate of our bloom filter algorithm, we evaluate the bloom filter on a held out set of passwords. We choose to use 20% of the passwords from the entire dataset to create our held out set. We then construct the bloom filter on the other 80% of the set and measure the false positive rate. For our set of 1 million passwords, we used 200,000 passwords for our held out set and 800,000 passwords to construct our bloom filter. With these parameters and a selected false positive rate of $p = 0.01$, the following bloom filter parameters were calculated using the above equations:

m (bit array size)	k (number of hash functions)
7,668,047	7

We observe the following after evaluating the accuracy of the bloom filter on held-out test set:

Expected FP Rate	Emperical FP Rate
1%	1.0035%

The empirical false positive rate and expected false positive rate are near identical, signifying that our implementation is correct.

Effect of k and m

We performed a set of ablation experiments on the bit array size (m) and the number of hash functions (k) and observed the change on the resulting false positive rate. In order to allow for high throughput on the experiments, we use smaller set of 50K passwords. We use the same train/test splitting strategy to test the false positive rate on a held out set (20% set of the entire set - 10K passwords). The bit array itself is built on the 40K set of passwords.

We vary m and k and observe the following:

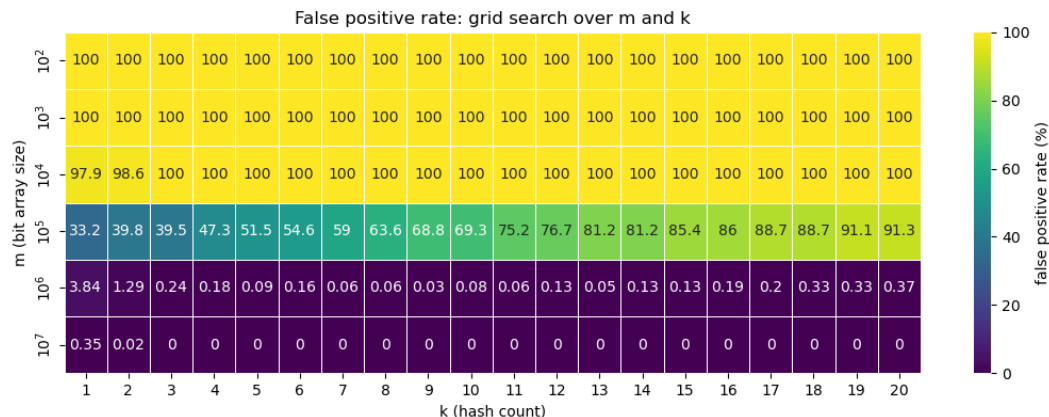


Figure 3: Heatmap of the False Positive rate of the 10K held out set for a varied m (bit array size) and k (hash count). FP rates are 100% when small bit array size is used and in most cases, 0% when a large bit array size is used with enough hash functions.

In the above heatmap, as we increase m , the false positive rate begins to decrease, no matter the value that we choose for k . This makes sense as increasing the bit array size would allow for more "expressiveness" as elements have a lower-probability of mapping to the same bits as another element.

Increasing k does not follow the same pattern: simply increasing k does not decrease the false positive rate. We see that there is a "sweet-spot" for when the false positive rate of the bloom filter is the lowest. Traversing in either direction of this sweet-spot results in a decrease in accuracy (this is most evident for $m = 10^6$).

Setting k and m very large, without maintaining proportionality, won't work because the hashing algorithm requires there to be a particular ratio between the two to be most effective. Furthermore, setting m to be very high would be counter-intuitive to the purpose of a bloom filter as the space complexity would massively increase. Even if the correct k was chosen, if m is too big, resulting the bit array is unable to fit into the cache, the empirical time-complexity of the bloom filter would massively increase as additional memory operations would be needed to dynamically transfer portions of the massive bit array into cache memory.

Query Time

To test the query time, the original 1M set of passwords is used to create the bloom filter using the same 80/20 split. We measure the time it takes to query the first 1000 passwords from the test set. We observe the following:

Overall Query Time (s) - 1000 Samples	Avg. Query Time (s) per Sample
0.00561 s	5.61×10^{-6} s

This is fast enough for most cases. IHaveBeenPwned.com's API would likely require something faster, or at least, have more processing power than the machine used for this test. For the intents and purposes of this project, it is certainly fast enough.

Memory

We use a Python byte array as the fundamental data-structure for bloom filter and use a traditional Python set for a baseline comparison. We observe the following difference in memory usage when constructing both the bloom filter and set on the train split of the 1M passwords set:

Bit Array - ($m = 7,668,047$)	Set - 80,000 elements
0.958 MB	8.389 MB

The bloom filter is far, *far* more memory efficient than the other python data structures to determine the presence of a particular value in a dataset (8.75x for space efficient). The byte array is a very dense, low-level data structure where we are explicitly managing individual bits (0s and 1s), sidestepping the heavy object overhead of Python's other data structure.

A Note on Deletion

Deletion is very messy with a bloom filter because of the overlap of multiple hashes. For example, if you add 'apple' and 'banana', they flip bits at indices at 2, 5, 9, and 1, 5, 22, respectively, to 1. Then you try to remove 'apple' you'd flip 2, 5, 9 back to 0. However, you have now corrupted 'banana' as well because it was also relying on 5 being 1. Our implementation does not know how many different items are sharing a specific index, so they cannot be safely turned back to 0. In a bloom filter, collisions are the intended compression mechanism so we would have to vastly change our implementation to allow deletion.

3 Conclusion

3.1 Three-Way Merge Sort

The three-way merge sort implementation provided a hands-on understanding of how divide-and-conquer sorting algorithms operate. During the first version, a critical case was overlooked: once one of the three sub-arrays is exhausted, the remaining two cannot both be drained simultaneously. Handling each pair of remaining sub-arrays independently was necessary to produce a correctly ordered final array.

The benchmark results aligned closely with theoretical expectations, confirming the $O(n \log n)$ asymptotic behavior of merge sort and serving as reinforcement of the Master Theorem. The test machine reached its limit at 2^{28} (268,435,456) elements, at which the operating system terminated the process due to memory exhaustion. As anticipated, floating-point inputs introduced slightly more overhead than integers, owing to the greater complexity of floating-point comparison and representation. Implementing the algorithm in a lower-level language such as C, where numeric types occupy fewer bytes and memory management is more direct, would likely allow significantly larger inputs to be sorted before hitting system limits.

3.2 Bloom Filter / Password Breach Checker

The bloom filter data structure presents a space and time efficient way to deterministically verify if elements do NOT have membership in a set and probabilistically determine which elements DO have membership. We re-create IHaveBeenPwned.com's functionality of determining which passwords have appeared in data breaches using a bloom filter on a subset of 1 million passwords from their full database.

Our validation confirms that the bloom filter provides a false positive rate identical to our 1% rate on a held-out set of 200K passwords. Ablation experiments show that the bloom filter performs best when the bit array size (m) and the number of independent hash functions (k) are proportional. Empirical query time analysis highlights the speed of the data structure while comparing the bit array data structure (basis of the bloom filter) to the traditional set data-structure results in the bloom filter being 8.75x more space efficient.

4 Contributions

Ricardo Garcia Implemented the 3-way merge sort algorithm, established the report outline, managed the team's Git repository and pull requests, and wrote the merge sort sections of the Introduction, Methods, and Conclusion.

Ethan Hoover Implemented the hashing algorithm, implemented the hashing algorithm tests, wrote the preliminaries and methods for hashing algorithm.

Shrey Poshia Implemented the password checker, performed validation experiments, wrote the bloom filter introduction, bloom validation results, and bloom filter conclusion.

Bibliography

- [1] AskPython. *How to get the Memory Size of Variables in Python?* <https://www.askpython.com/python/built-in-methods/variables-memory-size-in-python>. Accessed: 2026-04-05. 2023.
- [2] Troy Hunt. *Have I Been Pwned: Pwned Passwords*. <https://haveibeenpwned.com/Passwords>. Accessed: April 2026. 2024.